Republic of the Philippines
Bicol University Polangui
Polangui Albay

LAB REPORT

(CIRCULAR DOUBLY LINKED LIST)

SUBMITTED BY:

GIAN CARLO TRILLES
BScPE-2A

SUBMIITED TO:

MELISSA JEANKIE RELLON
INSTRUCTOR

DATE:
DECEMBER 09,2024

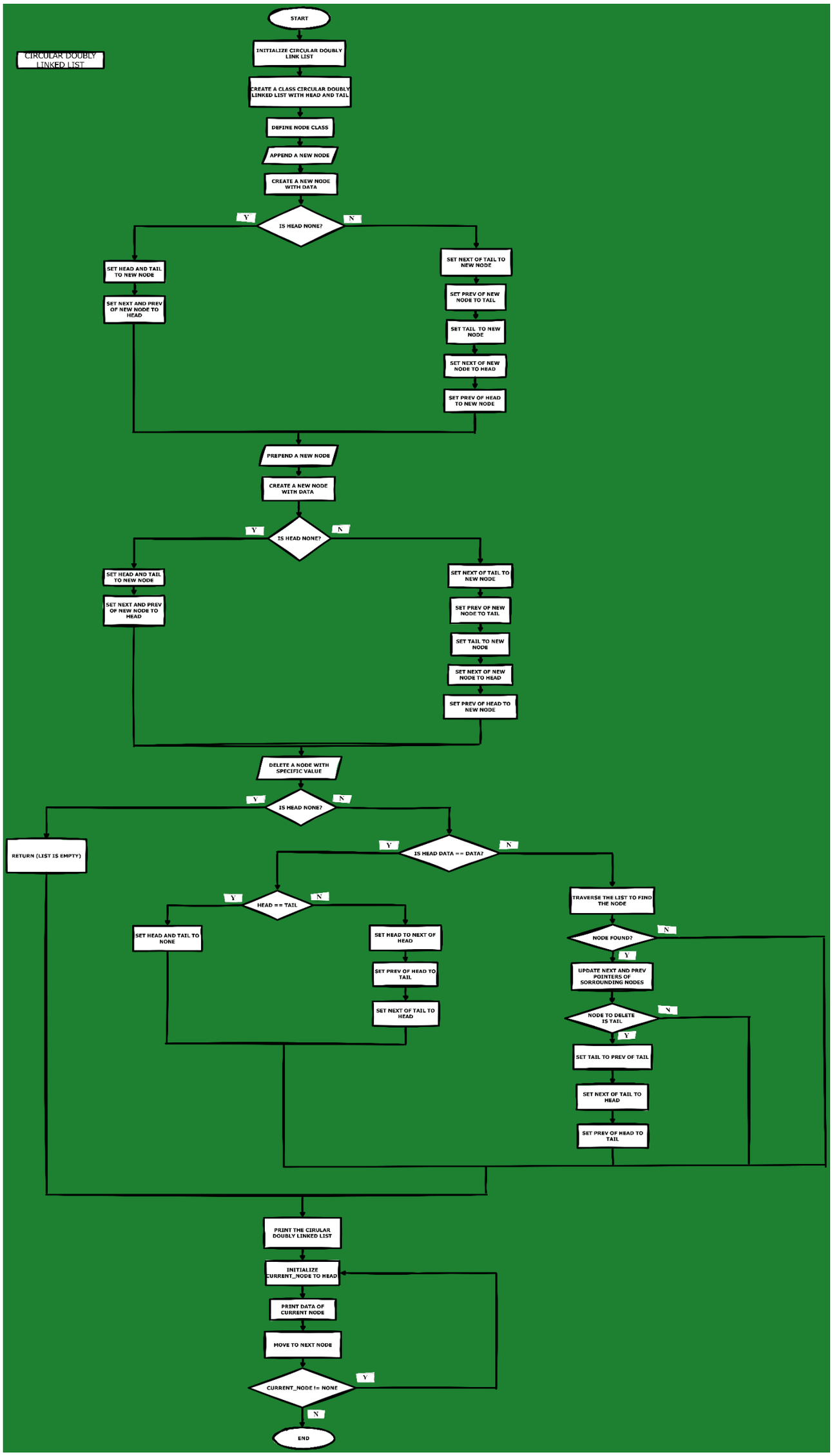
CIRCULAR DOUBLY LINKED LIST

MACHINE PROBLEM DESCRIPTION:

This program focuses on the operations of a Circular Doubly Linked List, a data structure where each node contains data, a reference to the next node, and a reference to the previous node. Unlike a regular Doubly Linked List, the last node in a Circular Doubly Linked List points back to the head node, and the head node points to the last node, forming a circular structure. Operations include adding nodes to the end (append), beginning (prepend), deleting nodes, and traversing the list to perform actions like printing data. This program helps us understand how Circular Doubly Linked Lists work and their applications in solving real-world problems, particularly where circular and bidirectional data structures are advantageous.

FLOWCHART (CIRCULAR DOUBLY LINKED LIST):

This flowchart represents a scenario where the program processes a Circular Doubly Linked List. The flowchart details the step-by-step process


CIRCULAR DOUBLY LINKED LIST
START
INITIALIZE CIRCULAR DOUBLY LINK LIST
CREATE A CLASS CIRCULAR DOUBLY LINKED LIST WITH HEAD AND TAIL
DEFINE NODE CLASS
APPEND A NEW NODE
CREATE A NEW NODE WITH DATA
IS HEAD NONE?
Y
N
SET HEAD AND TAIL TO NEW NODE
SET NEXT AND PREV OF NEW NODE TO HEAD
SET NEXT OF TAIL TO NEW NODE
SET PREV OF NEW NODE TO TAIL
SET TAIL TO NEW NODE
SET NEXT OF NEW NODE TO HEAD
SET PREV OF HEAD TO NEW NODE
PREPEND A NEW NODE
CREATE A NEW NODE WITH DATA
IS HEAD NONE?
Y
N
SET HEAD AND TAIL TO NEW NODE
SET NEXT AND PREV OF NEW NODE TO HEAD
SET NEXT OF TAIL TO NEW NODE
SET PREV OF NEW NODE TO TAIL
SET TAIL TO NEW NODE
SET NEXT OF NEW NODE TO HEAD
SET PREV OF HEAD TO NEW NODE
DELETE A NODE WITH SPECIFIC VALUE
IS HEAD NONE?
Y
N
RETURN (LIST IS EMPTY)
IS HEAD DATA == DATA?
Y
N
HEAD == TAIL
Y
N
SET HEAD AND TAIL TO NONE
SET HEAD TO NEXT OF HEAD
SET PREV OF HEAD TO TAIL
SET NEXT OF TAIL TO HEAD
TRAVERSE THE LIST TO FIND THE NODE
NODE FOUND?
N
Y
UPDATE NEXT AND PREV POINTERS OF SORROUNDING NODES
NODE TO DELETE IS TAIL
N
Y
SET TAIL TO PREV OF TAIL
SET NEXT OF TAIL TO HEAD
SET PREV OF HEAD TO TAIL
PRINT THE CIRULAR DOUBLY LINKED LIST
INITIALIZE CURRENT_NODE TO HEAD
PRINT DATA OF CURRENT NODE
MOVE TO NEXT NODE
CURRENT_NODE != NONE
Y
N
END

SOURCE CODE (CIRCULAR DOUBLY LINKED LIST):

The code below demonstrates the implementation of various operations on a Circular Doubly Linked List of integers.

```python
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None
        self.prev = None

class CircularDoublyLinkedList:
    def __init__(self):
        self.head = None
        self.tail = None

    def append(self, data):
        new_node = Node(data)
        if not self.head:
            self.head = new_node
            self.tail = new_node
            new_node.next = self.head
            new_node.prev = self.head
        else:
            self.tail.next = new_node
            new_node.prev = self.tail
            self.tail = new_node
            self.tail.next = self.head
            self.head.prev = self.tail

    def prepend(self, data):
        new_node = Node(data)
        if not self.head:
            self.head = new_node
            self.tail = new_node
            new_node.next = self.head
            new_node.prev = self.head
        else:
            new_node.next = self.head
            self.head.prev = new_node
            self.head = new_node
            self.head.prev = self.tail
            self.tail.next = self.head

    def delete(self, data):
        if not self.head:
            return "List is empty"

        if self.head.data == data:
            if self.head == self.tail:
                self.head = None
                self.tail = None
            else:
                self.head = self.head.next
                self.head.prev = self.tail
                self.tail.next = self.head
            return

        current = self.head
        while current.next != self.head:
            if current.next.data == data:
                current.next = current.next.next
                if current.next == self.head:
                    self.tail = current
                else:
                    current.next.prev = current
                return
            current = current.next

    def print_list(self):
        if not self.head:
            return "List is empty"

        current = self.head
        while True:
            print(current.data, end=" -> ")
            current = current.next
            if current == self.head:
                break
        print()
cdll = CircularDoublyLinkedList()
cdll.append(1)
cdll.append(2)
cdll.append(3)
cdll.prepend(0)
cdll.print_list()
cdll.delete(2)
cdll.print_list()
```

OUTPUT (CIRCULAR DOUBLY LINKED LIST):

The output displays the results of various operations on the Circular Doubly Linked List

```
[Running] python -u "c:\Users\Gian Carlo Trilles\OneDrive\Desktop\pythonfiles\CircularDoublyLinkList.py"
0 -> 1 -> 2 -> 3 ->
0 -> 1 -> 3 ->

[Done] exited with code=0 in 0.124 seconds
```